

Practical Assessment (Project-Based) — OU-ICT-WDDN-01-L4-V1: Apply Basic C#

Competency Standard: CS-ICT-WDDN-L4-EN-V1 **Unit:** Apply Basic C# | **Level:** 4 |

Nominal Hours: 30 **Assessment Type:** Project-Based Practical (single console application)

Duration: 2–3 hours (or take-home over 1–2 days) **Tools Required:** Visual Studio / VS Code, .NET SDK (Console App)

Project Brief: "Student Result Management Console System"

You have been hired to build a small console application for a training centre to manage student registrations, record marks, and generate result reports. The system must be built using only core C# fundamentals — no databases, no external libraries.

Build **ONE console project** named `StudentResultManager` that implements the features below as a single working menu-driven program. Every feature maps to a specific competency element — all must be present and working together, not as separate disconnected snippets.

Required Main Menu

```
==== Student Result Management System ====
1. Register New Student
2. Enter/Update Marks
3. View All Students
4. Search Student
5. Sort Students by Marks
6. Class Statistics (Highest/Lowest/Average)
7. Exit
```

The program must loop and keep showing this menu until the user selects **Exit** (Element 4 — loops).

Feature Requirements Mapped to Competency Elements

1. Register New Student → *Element 1: Work with Variables*

- Prompt for student **name** (`string`), **roll number** (`int`), and **date of registration**

(implicitly typed `var` using `DateTime.Now`).

- Follow proper C# naming conventions throughout.
- Store each student as an entry in a collection (see Element 6 below) — do not use separate loose variables per student.

2. Enter/Update Marks → *Element 2: Apply C# Basic Data Types*

- Marks for 3 subjects must be entered as `double`.
- Accept input as `string` from console and convert it using **both** `int.Parse`/`double.Parse` and `Convert.ToDouble` in different parts of the program.
- Calculate and display total and average, **formatted to 2 decimal places**.
- A student's `bool IsPassed` flag must be calculated (average ≥ 40 = true).

3. View All Students / Grade Assignment → *Element 3: Control Program Flow*

- Use `if-else` to assign a **grade** (A+, A, B, C, F) based on average marks.
- Use a `switch` statement somewhere in the menu logic to route the user's numeric menu choice (1-7) to the correct feature, with a `default` case for invalid input.

4. Menu Loop, Repeated Entry, Data Traversal → *Element 4: Work with Various Loops*

- Main menu must run inside a **do-while** loop (runs at least once, exits on option 7).
- Use a **for loop** to display students with their index numbers.
- Use a **while loop** with `break` to validate menu input (re-ask until valid number is entered).
- Use a **foreach loop** when printing the student list or calculating statistics.

5. Class Statistics → *Element 5: Work with Array*

- Store all students' **average marks** into an `array` (populated at runtime from your collection).
- Manually (without `Max()`/`Min()`/`Average()`) calculate and display the **highest**, **lowest**, and **class average** using array traversal.
- Bonus: represent a 3x3 "seating arrangement" of the first 9 students using a 2D array and print it as a grid.

6. Student Storage & Search → *Element 6: Work with Collection*

- Store all registered students in a `List<Student>` (define a simple `Student` class with Name, Roll, Marks fields).
- Implement **Search Student** using the roll number (linear search over the `List`).

- Use a `Dictionary<int, Student>` keyed by roll number as an alternative lookup for fast search, and compare its use to the `List` search in your code comments.
- Use a `Queue<string>` to simulate a "pending review" list — e.g., students who fail get queued for re-evaluation; dequeue and display them one by one.

7. Sort Students by Marks → *Element 7: Implement Data Structure and Algorithm*

- Implement **Bubble Sort manually** (no `List.Sort()`/`Array.Sort()`) to sort students by average marks, descending.
 - Implement **Binary Search manually** to find a student by roll number in the sorted list (after first sorting by roll number).
 - Print the number of comparisons each algorithm performs, and be ready to explain (orally) why Binary Search requires sorted data and its complexity vs Linear Search.
-

Deliverables

1. A single runnable console project (`.sln`/`.csproj` + source files).
 2. Program must run end-to-end without crashing on invalid input (e.g., entering text where a number is expected).
 3. Code should be reasonably commented, explaining which competency element each section demonstrates (e.g., `// Element 7: Bubble Sort implementation`).
 4. A short `README.txt` (5–10 lines) describing how to run the program and one design decision you made (e.g., why you used `List` vs `Dictionary`).
-

Assessor's Marking Guide

Feature Area	Competency Element(s)	Max Marks	Pass Mark
Register New Student	1. Variables	10	6
Enter/Update Marks & Conversion	2. Data Types	10	6
Grade Logic & Menu Routing	3. Control Flow	10	6
Menu Loop & Traversal	4. Loops	15	9
Class Statistics (Array)	5. Array	15	9

Feature Area	Competency Element(s)	Max Marks	Pass Mark
Storage & Search (List/Dictionary/Queue)	6. Collections	15	9
Sorting & Searching Algorithms	7. Data Structure/Algorithm	20	12
Overall Program Integration (runs end-to-end, handles bad input, is one cohesive app)	All	5	3
Total		100	60 (competent)

Oral Questioning (asked while candidate demonstrates the running project):

- Walk me through what happens when option 3 is selected — trace it through your `switch` statement.
- Why did you choose `List<Student>` over a plain array for storing students?
- What would break if you called Binary Search before sorting the list?
- Where in your project did you need type conversion, and why?

Required Attitude Observation (tick if demonstrated): ☐ Delivered ONE integrated project (not disconnected snippets) ☐ Code commented and readable ☐ Handled invalid input gracefully ☐ Could explain design choices when questioned ☐ Managed time appropriately across features

Context of Assessment: To be conducted at an NSDA-accredited assessment centre by an NSDA-certified/nominated assessor, per Unit of Competency OU-ICT-WDDN-01-L4-V1. Project-based format is acceptable under "Demonstration" and "Portfolio" methods of assessment listed in the Evidence Guide.